

PROGRAMACIÓN II

Trabajo Práctico 4: Programación Orientada a Objetos II

OBJETIVO GENERAL

Comprender y aplicar conceptos de Programación Orientada a Objetos en Java, incluyendo el uso de **this**, constructores, sobrecarga de métodos, encapsulamiento y miembros estáticos, para mejorar la modularidad, reutilización y diseño del código.

MARCO TEÓRICO

Concepto	Aplicación en el proyecto
Uso de this	Referencia a la instancia actual dentro de constructores y métodos
Constructores y sobrecarga	Inicialización flexible de objetos con múltiples formas de instanciación
Métodos sobrecargados	Definición de varias versiones de un método según los parámetros recibidos
toString()	Representación legible del estado de un objeto para visualización y depuración
Atributos estáticos	Variables compartidas por todas las instancias de una clase
Métodos estáticos	Funciones de clase invocadas sin instanciar objetos
Encapsulamiento	Restringir el acceso directo a los atributos de una clase

Caso Práctico

Sistema de Gestión de Empleados

Modelar una clase **Empleado** que represente a un trabajador en una empresa. Esta clase debe incluir constructores sobrecargados, métodos sobrecargados y el uso de atributos aplicando encapsulamiento y métodos estáticos para llevar control de los objetos creados.

CLASE EMPLEADO

Atributos:

- **int id**: Identificador único del empleado.
- **String nombre**: Nombre completo.
- **String puesto**: Cargo que desempeña.
- **double salario**: Salario actual.
- **static int totalEmpleados**: Contador global de empleados creados.

REQUERIMIENTOS

1. Uso de **this**:
 - Utilizar **this** en los constructores para distinguir parámetros de atributos.
2. Constructores sobrecargados:
 - Uno que reciba todos los atributos como parámetros.
 - Otro que reciba solo nombre y puesto, asignando un id automático y un salario por defecto.
 - Ambos deben incrementar **totalEmpleados**.
3. Métodos sobrecargados **actualizarSalario**:
 - Uno que reciba un porcentaje de aumento.
 - Otro que reciba una cantidad fija a aumentar.
4. Método **toString()**:
 - Mostrar id, nombre, puesto y salario de forma legible.
5. Método estático **mostrarTotalEmpleados()**:
 - Retornar el total de empleados creados hasta el momento.
6. Encapsulamiento en los atributos:
 - Restringir el acceso directo a los atributos de la clase.
 - Crear los métodos Getters y Setters correspondientes.

TAREAS A REALIZAR

1. Implementar la clase Empleado aplicando todos los puntos anteriores.
2. Crear una clase de prueba con método main que:
 - Instancie varios objetos usando ambos constructores.
 - Aplique los métodos **actualizarSalario()** sobre distintos empleados.
 - Imprima la información de cada empleado con **toString()**.
 - Muestre el total de empleados creados con **mostrarTotalEmpleados()**.

CONSEJOS

- Usá **this** en los constructores para evitar errores de asignación.
- Probá distintos escenarios para validar el comportamiento de los métodos sobrecargados.
- Asegurate de que el método **toString()** sea claro y útil para depuración.
- Confirmá que el contador **totalEmpleados** se actualiza correctamente en cada constructor.

CONCLUSIONES ESPERADAS

- Comprender el uso de **this** para acceder a atributos de instancia.
- Aplicar constructores sobrecargados para flexibilizar la creación de objetos.
- Aplicar encapsulamiento en los atributos.
- Implementar métodos con el mismo nombre y distintos parámetros.
- Representar objetos con **toString()** para mejorar la depuración.
- Diferenciar y aplicar atributos y métodos estáticos en Java.
- Reforzar el diseño modular y reutilizable mediante el paradigma orientado a objetos.

Capturas:

```
package com.mycompany.trabajo.practico.poo;

public class TRAJOPRACTICOPOO {

    public static void main(String[] args) {
        Empleado empleado1 = new Empleado(1, "Lorenzo", "Administrativo", 1100000.00);
        System.out.println("Empleado " + empleado1);
        System.out.println("Total de empleados: " + empleado1.getTotalEmpleados());

        Empleado empleado2 = new Empleado(1, "Joan", "Administrativo", 1100000.00);
        System.out.println("Empleado " + empleado2);
        System.out.println("Total de empleados: " + empleado1.getTotalEmpleados());

        Empleado empleado3 = new Empleado(1, "Belen", "Vendendor", 600000.00);
        System.out.println("Empleado " + empleado3);
        System.out.println("Total de empleados: " + empleado1.getTotalEmpleados());

        System.out.println("Actualización de salarios: ");

        empleado1.actualizarSalario(100.0);
        System.out.println("Empleado " + empleado1);

        empleado2.actualizarSalario(1.0);
        System.out.println("Empleado " + empleado2);

        empleado3.actualizarSalario(1000.0);
        System.out.println("Empleado " + empleado3);
    }
}
```

```
package com.mycompany.trabajo.practico.poo;

public class Empleado {
    private int id;
    private String nombre;
    private String puesto;
    private Double salario;
    private static int totalEmpleados;

    //Constructor.
    public Empleado(int id, String nombre, String puesto, Double salario) {
        this.id = id;
        this.nombre = nombre;
        this.puesto = puesto;
        this.salario = salario;
        totalEmpleados ++;
    }

    //Sobrecarga de constructor.
    public Empleado(String nombre, String puesto) {
        this(10, nombre, puesto, 500.00);
    }
}
```

```
//Getters y Setters.  
public int getId() {  
    return id;  
}  
  
public void setId(int id) {  
    this.id = id;  
}  
  
public String getNombre() {  
    return nombre;  
}  
  
public void setNombre(String nombre) {  
    this.nombre = nombre;  
}  
  
public String getPuesto() {  
    return puesto;  
}  
  
public void setPuesto(String puesto) {  
    this.puesto = puesto;  
}  
  
public Double getSalario() {  
    return salario;  
}  
  
public static int getTotalEmpleados() {  
    return totalEmpleados;  
}
```

```
public void actualizarSalario(int porcentaje) {  
    if (porcentaje >= 0) {  
        this.salario *= (1 + porcentaje / 100.0);  
    }  
}  
  
public void actualizarSalario(double cantidad) {  
    if (cantidad >= 0) {  
        this.salario += cantidad;  
    }  
}  
  
public static void setTotalEmpleados(int totalEmpleados) {  
    Empleados.totalEmpleados = totalEmpleados;  
}  
  
@Override  
public String toString() {  
    return "Empleados{" + "id = " + id + ", nombre = " + nombre + ", puesto = " + puesto + ", salario = " + salario + "}";  
}
```

```
--- exec:3.1.0:exec (default-cli) @ TRABAJO-PRACTICO-POO ---
Empleado Empleado{id = 1, nombre = Lorenzo, puesto = Administrativo, salario = $1100000.0}
Total de empleados: 1
Empleado Empleado{id = 1, nombre = Joan, puesto = Administrativo, salario = $1100000.0}
Total de empleados: 2
Empleado Empleado{id = 1, nombre = Belen, puesto = Vendendor, salario = $600000.0}
Total de empleados: 3
Actualización de salarios:
Empleado Empleado{id = 1, nombre = Lorenzo, puesto = Administrativo, salario = $1100100.0}
Empleado Empleado{id = 1, nombre = Joan, puesto = Administrativo, salario = $1100001.0}
Empleado Empleado{id = 1, nombre = Belen, puesto = Vendendor, salario = $601000.0}
-----
BUILD SUCCESS
-----
```